

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA10	Course Title:	Software Engineering
CO Assessed:	CO4	Assessment:	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	Total Marks:	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	Pranayaa S	Reg. No.:	192524110
Date:	14-08-2026	Duration:	60 minutes

Code of Conduct

I Pranayaa S (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Pranayaa S _____

Annexure A – CI/CD Pipeline Design Exercise

Instructions to Students:

Each student is assigned an individual problem statement. Based on the assigned problem statement, **design a CI/CD (Continuous Integration and Continuous Deployment) pipeline** for the proposed software system to automate the processes of code integration, building, testing, and deployment.

The exercise should include:

1. **Analyze the project requirements** and identify the CI/CD needs of the assigned problem statement.
2. **Design the CI/CD pipeline workflow** showing stages such as source code management, build, testing, code quality analysis, packaging, and deployment.
3. Identify suitable **tools and technologies** for each stage of the pipeline and justify their selection.
4. Define appropriate **automated testing strategies** to be integrated into the pipeline.
5. Design the **deployment strategy**, including development, testing/staging, and production environments.
6. Incorporate appropriate **security, quality checks, notifications, and rollback mechanisms** in the pipeline.
7. Prepare a **CI/CD pipeline diagram** and explain the workflow from code commit to deployment.

Deliverable: Submit a **CI/CD Pipeline Design document** containing the pipeline architecture/diagram, stages, tools, configuration/workflow, testing strategy, deployment process, and justification based on the assigned problem statement.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Pipeline Analysis(15%)	Clearly analyzes the assigned problem and identifies comprehensive CI/CD requirements	Identifies most relevant requirements	Identifies basic requirements with some gaps	Requirements are unclear or incomplete
Pipeline Architecture & Workflow(25 %)	Designs a complete, logical pipeline covering source, build, test, quality check, package, and deployment stages	Pipeline covers most major stages with minor gaps	Basic pipeline is designed but several stages are missing	Pipeline is incomplete or poorly structured
Tools & Technology Selection(20%)	Selects appropriate tools for each stage and provides clear justification	Appropriate tools selected with reasonable justification	Tools are identified but justification is limited	Tools are inappropriate or not clearly identified
Automated Testing & Quality Integration(15 %)	Effectively integrates automated testing and code-quality/security checks into the pipeline	Includes major testing and quality checks	Includes basic testing with limited integration	Testing/quality checks are missing or inappropriate
Deployment, Security & Rollback Strategy(15%)	Clearly designs deployment environments, security practices, monitoring, and rollback strategy	Covers deployment and most security/rollback aspects	Basic deployment strategy with limited security/rollback details	Deployment strategy is incomplete or lacks security considerations
Pipeline Diagram & Documentation (10%)	Clear, accurate, professional diagram with excellent explanation and documentation	Well-presented diagram and documentation with minor issues	Adequate diagram/documentation but lacks clarity	Poorly presented, incomplete, or difficult to understand

Criteria	Mark
Requirement & Pipeline Analysis(15%)	/5
Pipeline Architecture & Workflow(25%)	/4
Tools & Technology Selection(20%)	/4
Automated Testing & Quality Integration(15%)	/4
Deployment, Security & Rollback Strategy(15%)	/4
Pipeline Diagram & Documentation(10%)	/4
TOTAL	/25

1. Requirement & Pipeline Analysis

The automated deployment pipeline for the AI-Based Smart Retail Inventory Forecasting and Demand Planning System must facilitate rapid, secure, and reliable integration of machine learning models and full-stack application code. Key requirements include automated linting, unit testing, security vulnerability scanning, Docker container image building, and multi-environment deployment.

In addition to standard deployment workflows, the pipeline incorporates automated database migration checks using Alembic. Prior to container promotion, database rollback scripts are validated against a staging replica to ensure zero data loss during schema evolution. Slack and PagerDuty notification webhooks are integrated at each pipeline boundary to provide real-time status alerts to engineering teams.

2. Pipeline Architecture & Workflow Design

The CI/CD workflow follows a sequential, multi-stage pipeline designed to ensure code quality and stability before production rollout:

1. Source Code Management (SCM): Triggers on code push or pull request to GitHub repository.
2. Build & Lint: Compiles frontend assets and verifies Python/TypeScript syntax.
3. Automated Testing: Executes pytest unit tests and integration test suites.
4. Code Quality & Security Scans: Performs static analysis via SonarQube and dependency vulnerability audits via Trivy.
5. Container Packaging: Builds multi-architecture Docker images and pushes them to a secure container registry.
6. Deployment: Automated rollout to Staging and Production Kubernetes clusters.

In addition to standard deployment workflows, the pipeline incorporates automated database migration checks using Alembic. Prior to container promotion, database rollback scripts are validated against a staging replica to ensure zero data loss during schema evolution. Slack and PagerDuty notification webhooks are integrated at each pipeline boundary to provide real-time status alerts to engineering teams.

3. Tools and Technology Selection

- CI/CD Orchestration: GitHub Actions for native repository integration and workflow flexibility.
- Containerization: Docker & Docker Compose for consistent local and production environments.
- Code Quality & Security: SonarQube for static analysis; Trivy for container vulnerability scanning.
- Cloud Infrastructure: Kubernetes (Amazon EKS) for scalable container orchestration in production.

In addition to standard deployment workflows, the pipeline incorporates automated database migration checks using Alembic. Prior to container promotion, database rollback scripts are validated against a

staging replica to ensure zero data loss during schema evolution. Slack and PagerDuty notification webhooks are integrated at each pipeline boundary to provide real-time status alerts to engineering teams.

4. Automated Testing, Security & Rollback Strategy

Unit and integration test suites execute automatically on every pull request. Code quality gates block merges if test coverage drops below 85% or if SonarQube detects critical security vulnerabilities.

Deployment environments are isolated into Development, Staging, and Production. Secrets are managed via encrypted GitHub Secrets and cloud parameter stores. In the event of a deployment failure, if HTTP 5xx error rates exceed 2% within 5 minutes, Kubernetes initiates an automatic rollback to the previous stable pod revision.

In addition to standard deployment workflows, the pipeline incorporates automated database migration checks using Alembic. Prior to container promotion, database rollback scripts are validated against a staging replica to ensure zero data loss during schema evolution. Slack and PagerDuty notification webhooks are integrated at each pipeline boundary to provide real-time status alerts to engineering teams.